FL-IDS, explained simply

What we built, why it matters, how we pushed accuracy past 95%, and what is new about it.

Our project is a **security guard for networks of smart devices**. It learns from many places at once without peeking at their private data, it can't easily be tricked during training, it notices troublemakers it has never seen before, and when it spots one it locks the door on them automatically.

FL-IDS: Federated Learning-Based Intrusion Detection System for IoT/IIoT · Aaron B Ajay, Aaryan Prakash, Amal Roy, Amogh Krishna JS · Guide: Mr. Syed Akram · Dept. of CSE, BMSCE

1. The problem

Factories, hospitals, power plants and smart buildings are full of small connected gadgets: temperature sensors, water-level meters, smart controllers. This is the **Internet of Things (IoT)**, and in factories, **Industrial IoT (IIoT)**. These gadgets talk to each other all day by sending tiny messages called **packets**.

Bad people attack these networks: they flood them with junk (DDoS), sneak into web pages (SQL injection, XSS), guess passwords, plant hidden back doors and ransomware, or listen in the middle of conversations (MITM). An **intrusion detection system (IDS)** watches the packets and shouts when something is wrong.

Like you're five: Imagine a school with many classrooms. Lots of notes get passed around all day. Most are normal (“see you at lunch”), but a few are from bullies. We want a hall monitor who can read the notes and catch the bullies.

Ordinary hall monitors have four problems:

Privacy

To learn, they collect every note from every classroom into one big pile. Schools don't want to hand over private notes.

New bullies

They only know the bullies in their photo book. A brand-new bully walks right past.

Sabotage

If many classrooms help train the monitor, one sneaky classroom can teach it wrong things on purpose.

Shouting isn't stopping

Spotting a bully is not enough. Someone still has to close the door.

2. Our solution: five helpers working together

1 The guard with a photo book (LightGBM classifier)

It has studied photos of 14 kinds of known troublemakers. When a packet comes in, it says “that's a password attack” or “that's an HTTP flood”. If it is at least 70% sure, that's the answer.

The grown-up version: LightGBM builds hundreds of small yes/no decision trees and adds up their votes. It is very good at tables of numbers like packet fields.

2 The guard who knows what “normal” looks like (autoencoder)

This guard has never seen a bully's photo. Instead it has watched normal notes so much that it can tell when something looks *weird*. If a note is too different from normal, it says “suspicious!”, even for a bully nobody has seen before. These brand-new attacks are called **zero-day attacks**.

The grown-up version: An autoencoder squeezes each packet's 91 numbers down to 8 and tries to rebuild them. It rebuilds normal traffic well and strange traffic badly; a big rebuilding error means “anomalous”. It only checks what guard 1 was not sure about.

3 Learning together without sharing secrets (federated learning)

Each classroom trains its own copy of guard 2 using only its own notes. Then it sends just *what it learned* (not the notes!) to the principal, who combines everyone's lessons into one better guard and sends it back. Repeat 20 times.

The grown-up version: Federated learning with the Flower framework: 10 “clients” (sites) and one server. Only model weight changes travel; raw traffic stays on each site.

4 The fair principal (trust filter)

What if one classroom is sneaky and sends wrong lessons on purpose? The principal compares every classroom's lesson with what most classrooms said. A lesson pointing in the *opposite* direction, a very odd one, or one from a classroom that keeps disagreeing round after round is thrown away. Very loud lessons are turned down so no one dominates, and the principal averages the rest while ignoring the most extreme answers.

The grown-up version: Cosine similarity of each update against the median update, a MAD outlier test, a running trust score, norm clipping and a trimmed mean.

5 The automatic door (software-defined networking, SDN)

When a bully is caught, the system tells the network switch, which is like the school's door controller: “don't let this one in any more.” The door locks by itself.

The grown-up version: An OpenFlow controller (os-ken) installs a “drop” rule on an Open vSwitch switch inside an emulated network (Mininet). We read the rule back from the switch to prove it was really installed.

How a packet is judged

Packet arrives plus clues about its whole conversation → **Guard 1** “Known attack, and I'm sure?” → name it → **Guard 2** not sure? “Does it look normal?” → **Verdict** attack type, “suspicious”, or normal → **Door** attacker blocked on the switch

3. The big problem we had: accuracy

Our goal was **at least 95% accuracy and at least 0.95 macro F1**. Before this semester's work we had **96.4% accuracy** but only **0.889 macro F1**. So why wasn't 96.4% good enough?

Like you're five: Imagine 100 notes, and 71 of them are normal. A lazy monitor who says “everything is fine” to every single note is already right 71 times out of 100, yet catches zero bullies! So “accuracy” can look good while the important part is bad. **Macro F1** fixes this: it gives each kind of bully its own score and averages them, so rare bullies count just as much as common ones.

Why it was stuck: the empty envelopes

The attacks we kept mixing up were the **web attacks**: SQL injection, XSS (sneaking scripts into web pages), uploading bad files, password guessing and HTTP floods. When we looked closely, we found something surprising:

Like you're five: Most of the notes in these attacks were **empty envelopes** that just said “got it”. Only 5 to 17 out of every 100 notes actually had a message inside. An empty “got it” envelope looks exactly the same whichever bully sent it, so *nobody* could tell them apart, however clever.

The grown-up version: In those classes only 5–17% of packets carry HTTP content; the rest are TCP acknowledgements to port 80 on the same web server. With the original 91 features, 2.31% of all rows had an identical twin with a *different* label. That caps macro recall at 0.969 for any per-packet model. The useful information was in columns the dataset's standard recipe throws away.

4. What we changed, and why each change helps

1 Which door?

Which service the conversation is for: the website door (port 80), the sensor-messaging door (1883), and so on.

Why: tells web traffic apart from sensor traffic.

2 What's in the letter?

For notes that do have a message: how long it is, how jumbled, and whether it contains tell-tale words like “SELECT” (database trick), “<script>” (web trick), file uploads or logins.

Why: each web attack writes different things.

3 The whole conversation

We group notes into conversations, and every note, even an empty envelope, gets told what its whole conversation looked like: how many notes, how fast, what words appeared.

Why: now each empty envelope carries its conversation's story.

+ Rare bullies count more

Some attacks are very rare (Fingerprinting has only about 1,000 packets out of 2 million). We told the model to pay a bit more attention to rare kinds (square-root class weights), so it doesn't ignore them.

Each change, measured on conversations the model never saw. The big jump comes from the conversation clues.

What the model knew	Accuracy	Macro F1
Original 91 features	96.4%	0.889
+ which door (service port)	98.0%	0.924
+ what's in the letter (content)	98.0%	0.923
+ the whole conversation (flow)	99.75%	0.981
+ rare classes weighted (final)	**99.7%**	**0.983**

Strict test, seed 42. The content clues add little on their own because few packets carry content, but they become powerful once they are shared across the whole conversation.

5. How we made sure we weren't cheating

A model can get high scores by “cheating”, for example by memorising answers instead of learning. We checked for this carefully.

A strict test

Normally people test on random notes, so notes from the same conversation end up in both the study pile and the exam. That's like seeing half the exam answers while studying. We kept **whole conversations** out of studying, so the exam is truly new. We also ran it three times with different random shuffles: macro F1 was 0.983, 0.983 and 0.985 every time.

We fixed a cheat sheet hidden in the public dataset

In the dataset everyone uses, “nothing here” was written as `0` in normal traffic and as `0.0` in attack traffic. That tiny spelling difference gave away the answer perfectly! We made both spellings the same. (We spotted this ourselves, but a researcher called Galal published it first, in August 2026, so the credit is theirs.) We also found that the MITM attack rows were in the wrong columns, and rebuilt them from the original recordings. Nobody had reported that one.

No peeking at name tags

We never let the model see IP addresses or times. In this dataset each attack was recorded separately, so those would tell the model the answer without it learning anything.

A mistake we caught

Our first strict test went wrong: all the normal “remote screen” (VNC) traffic was *one* hour-long conversation, so it landed entirely in the exam and the model had never seen anything like it. We fixed this the standard way network tools do: very long conversations are cut into 2-minute pieces.

Fair with less data too

In our real system, the principal only keeps a small labelled set, 5% of the data. Trained on just that 5%, the classifier still scores 99.7% accuracy and 0.979 macro F1.

6. Results, in plain words

99.7%	0.983	0.94	40%
accuracy (target 95%)	macro F1 (target 0.95)	catches brand-new attacks (0.80 without guard 2)	bad classrooms it can survive

The confusion matrix: a scoreboard of who got mistaken for whom

Each **row** is the real kind of traffic; each **column** is what the model guessed. If the model were perfect, only the diagonal squares (top-left to bottom-right) would be dark.

Ours is almost all diagonal. 11 of the 14 kinds score 0.98–1.00. The only small mix-ups:

- **Password ↔ HTTP flood**: both knock on the same website door.
- **Ransomware → Backdoor**: both use the same door number.

MITM is missing here because all of it is a single conversation, so it can't be split between studying and the exam. On the normal test it scores 0.99.

Catching bullies nobody has seen before

We hid each attack type completely from training, one at a time, then checked if it was still caught. With the new clues, guard 1 alone caught 80% on average, and with guard 2 helping, **94%**. Before the new clues it was 53% and 74%. The web bullies improved the most: Password went from 44% to 97%, XSS from 52% to 99%. One got worse: MITM dropped from 89% to 69%, because its single conversation now looks quite ordinary to guard 2. We report that instead of hiding it.

Surviving sabotage

We made some classrooms send poisoned lessons on purpose. With the fair principal, known bullies were still caught almost 100% of the time, and brand-new bullies over 90% of the time, even with **40%** bad classrooms. Without the principal, plain averaging broke at 30%. At 50% the bad classrooms are the majority, so “what most classrooms say” becomes their lie. That is a known limit of this kind of defence.

The live demo

Replay
a real recorded backdoor attack (24,914 packets)
→
Detect
“Backdoor”, 99.7% sure
→
Decide
sure enough → block
→
Block
drop rule on the switch, checked on the switch itself

We also ran the real thing with ten separate programs as ten sites: it caught 0.999 of attacks (0.889 before the new clues) with 3.5% false alarms, the same as our faster simulations.

7. What is new (novel) about this project?

All five helpers in one system

Other researchers have built pieces: federated anomaly detectors, trust filters, two-stage classifiers, or federated learning with SDN. In our searches we found no published system combining all five: the photo-book guard, the “looks normal?” guard, learning together, the fair principal and the automatic door. We measured exactly how much each part adds.

Honest note: this is based on targeted searches, not a full systematic review, so we say “we found none”, not “none exist”.

We found an error in a popular public dataset

The broken MITM rows are our own find. Together with the “0 vs 0.0” cheat sheet (first reported by Galal, 2026), they mean other papers using this dataset may report scores that are too high. That is a useful finding for the whole research community.

Accuracy reached the honest way

We reached 99.7% / 0.983 by giving the model information that really describes attacks (which door, what was sent, how the conversation behaved), tested on conversations it never saw, without using names, times or the dataset's hidden cheat.

8. What it can't do yet (honest limits)

- **The door locks after the recording is analysed**, not packet by packet while the attack is happening.
- **The “which door” clue is partly a testbed effect.** In this dataset, normal traffic never uses the website door, so “website door” already hints at an attack. In a real office with normal web browsing this clue would help less.
- **Guard 2 makes a few false alarms on purpose** (about 3 in 100 friendly notes). For bullies guard 1 already knows, that costs a little; guard 2 earns its place on brand-new bullies.
- Two kinds of saboteur were tested. The second, a clever one that knows how our guard works, **slips past it most of the time**, and the defence fails at 50% bad sites.

9. What we did next

A real 20-school test, and more tries

Before, our 20-school test was pretend (everything in one program). Now we ran it for real: 20 separate schools plus the principal, on a bigger computer. We also repeated the big tests with three different shuffles of the data. Almost everything came out the same each time, but a few numbers wobble (like how many new bullies the second guard catches alone), so we give ranges, not one number.

Like you're five: Doing a test three times with different shuffles is like asking three different friends to try your game. If they all win about the same, you can trust the game a bit more.

One odd score, explained

One 20-school run scored a little lower. The reason: of the tiny group of “Fingerprinting” notes it was tested on, there were only 4, and it missed 1. It was not a hidden problem.

A cleverer saboteur

The sneaky school that knows exactly how the principal's filter works now gets through most of the time (72–90% of its notes are accepted, against 10–36% for the simple saboteur). Known bullies are still caught by guard 1, so the harm stays small, but with 4 sneaky schools out of 10, guard 2 on its own catches only 50 in 100 new-bully notes instead of 66. We did not test new bullies against this sneaky school, so we say so.

The grown-up version: A white-box attacker that keeps cosine similarity 0.1 to the last aggregate update and hides its harm in the orthogonal direction. Attacker survival 0.72–0.90; autoencoder-alone recall 0.499 at 40% attackers.

Making the model smaller (pruning)

We tried deleting the smallest 70% of the numbers inside guard 2's brain. If we delete them *while it learns*, it adapts and barely gets worse (catches 69 of 100 new-bully notes instead of 74). If we delete them *after* it has learned, it forgets (27 of 100 at half deleted). So “prune while training” is the way. The schools still send full-size messages, so this makes the brain smaller but does not shrink the mail.

A stricter exam: earlier notes teach, later notes test

We also trained on the earlier part of each kind of traffic and tested on the later part. Overall accuracy stays 99.4%. The score for “average over kinds” looks lower (0.913) only because MITM is one single conversation, so all of it lands in the test and the model never saw any; without that, it is about 0.978. Backdoor and website-flood notes did get a little harder.

10. Mini glossary

Packet	One small message on a network, like one note.
Flow / connection	All the packets of one conversation between two devices.
Classifier	A model that puts things into named groups (“SQL injection”, “normal”…).
Autoencoder	A model that learns what normal looks like and flags things it can't rebuild well.
Zero-day attack	An attack type the system was never trained on.
Federated learning	Training one model across many sites by sharing what was learned, not the data.
Poisoning	A bad participant sending harmful updates on purpose during training.
Trust filter	Our check that drops updates pointing against the group's consensus.
SDN / OpenFlow	Networks whose switches can be told new rules by software, e.g. “drop this sender”.
Accuracy	Share of all answers that are right.
Macro F1	Average score over every class, so rare attacks count as much as common ones.
Confusion matrix	A table of real class vs. predicted class; the diagonal is “right”.